

K-anonymity, using Clingo to anonymize data sources

nr:837377516

Wilfred Van Casteren

Open Universiteit

Masteropleiding AI

Model-based AI

December 12, 2024

Abstract

In this writing I will propose a potential usage of Answer Set Programming when anonymizing datasources. The solution uses the Python Clingo api. The final product will be a proof of concept. It will be accompanied by a file which facilitates checking the search part of the general idea proposed here.

Contents

1	Open dataset and conversion to asp-statements	3
1.1	Data	3
1.2	User data	4
2	Towards a solution	5
2.1	General idea	5
2.2	Generate column priority counts and all possible priority and sub-priority numbers	5
2.2.1	Clingo-file	6
2.3	Generate column combinations	6
2.3.1	Clingo file	8
2.3.2	Facts generated	8
2.3.3	Remark	9
2.4	Add a deletion rank to full priority numbers	10
2.4.1	Clingo-file	10
2.4.2	Facts generated	11
3	Searching for the solution	12
3.1	Purpose	12
3.2	Remarks	12
3.3	Searching	12
3.3.1	Clingo file	13
3.3.2	Data specific files	13
3.3.3	col-specific-hide.lp	14
3.3.4	search-constraints.lp	15
3.3.5	Facts generated	16
3.4	Export to csv	16
3.4.1	New files referred to	17

Development environment

To reach a solution I have used PyCharm, the Python Clingo api is very easy to use. When using more advanced features some things weren't that easy and/or scarcely documented. For creation of Clingo files I turned to Visual Studio Code, which has some extensions for Clingo development. [Figure 1](#) shows the PyCharm environment, to the left one can see the multitude of generated and handwritten Clingo files.

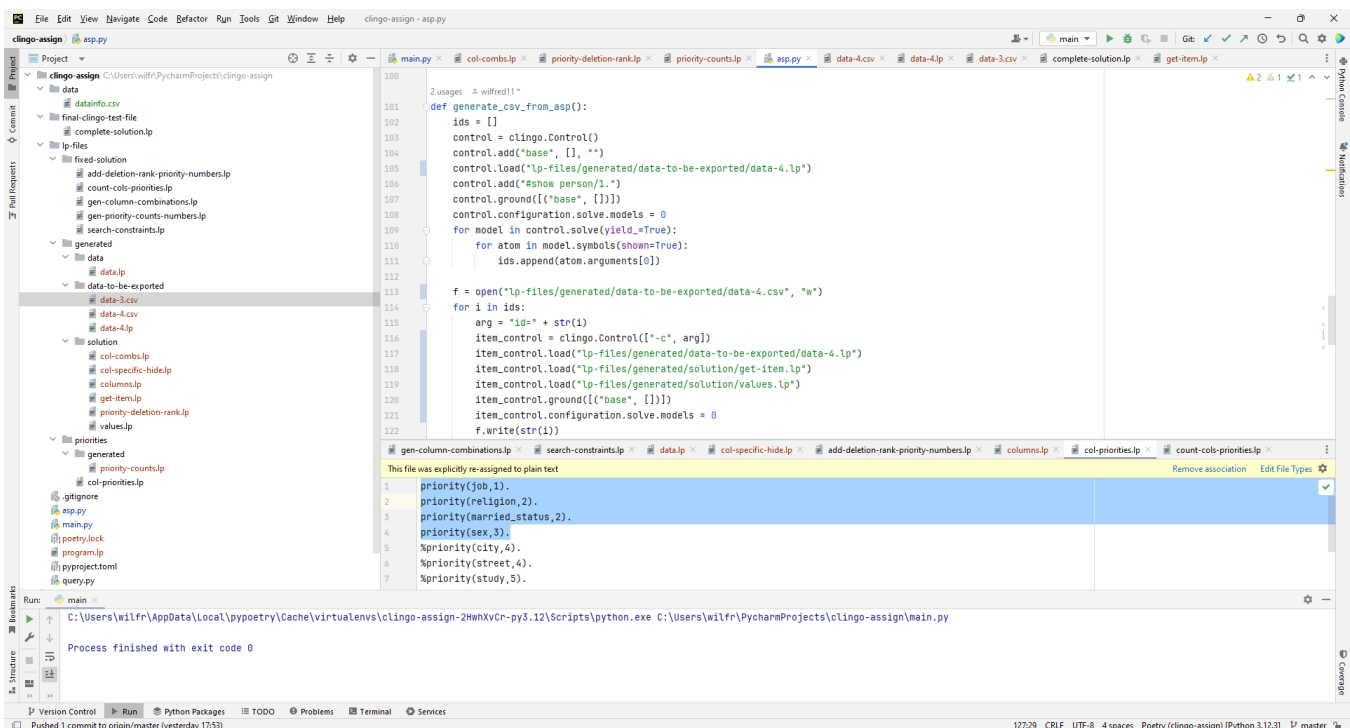


Figure 1: PyCharm.

Remarks

In my solution which is a combination of Clingo files and Python, I will sometimes make some minor adaptations to generated files using Python code. But the main work will be done by the Clingo files.

I tested my solution only with a small dataset and didn't use any asynchronous code.

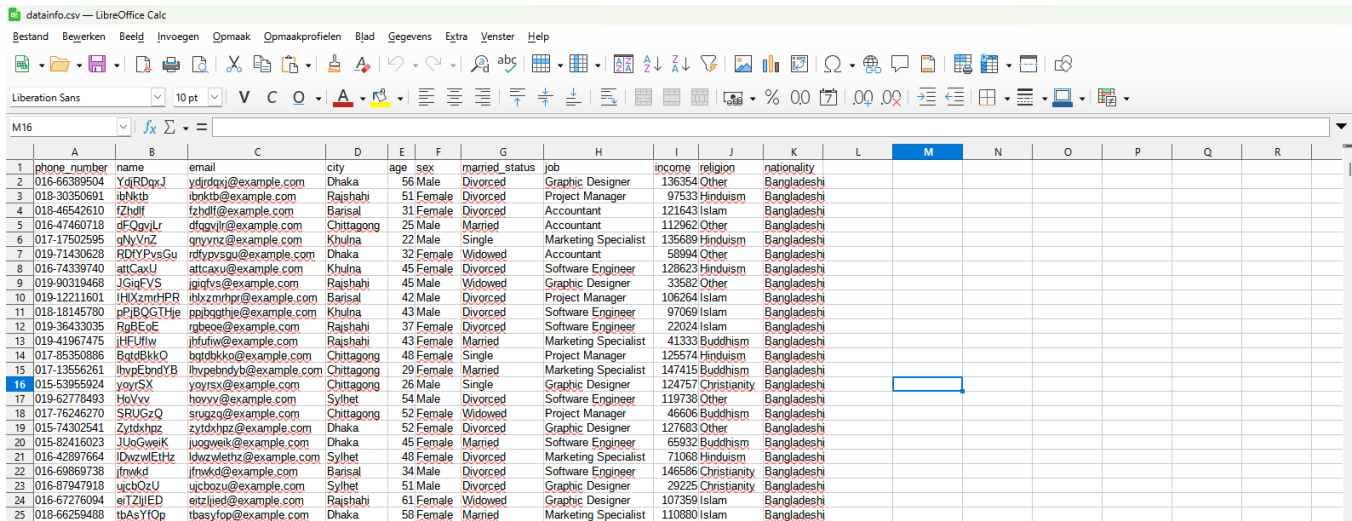
It would be possible to generate a solution without or using less intermediate results, but at the moment this way of working makes it easier to check the results of different parts of the solution.

Chapter 1

Open dataset and conversion to asp-statements

1.1 Data

For the data I used a fake dataset, that was found somewhere on the internet. I only used 4 columns of the data.



	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
1	phone_number	name	email	city	age	sex	married_status	job	income	religion	nationality							
2	016-66389504	YqfRDqxJ	ydrdpxj@example.com	Dhaka	56	Male	Divorced	Graphic Designer	136354	Other	Bangladeshi							
3	018-30350691	ibNkIb	ibnktb@example.com	Rajshahi	51	Female	Divorced	Project Manager	97533	Hinduism	Bangladeshi							
4	018-47642610	fzhdlf	fzhdlf@example.com	Barisal	31	Female	Divorced	Accountant	121643	Islam	Bangladeshi							
5	016-47460718	dFQqvlr	dfoqvlr@example.com	Chittagong	25	Male	Married	Accountant	112962	Other	Bangladeshi							
6	017-17502595	qNyVnZ	qnyvnrz@example.com	Khulna	22	Male	Single	Marketing Specialist	135689	Hinduism	Bangladeshi							
7	019-71430628	RdTYPvsGu	rdtypvsgu@example.com	Dhaka	32	Female	Widowed	Accountant	58994	Other	Bangladeshi							
8	016-74339740	atCaxU	attcaxu@example.com	Khulna	45	Female	Divorced	Software Engineer	128623	Hinduism	Bangladeshi							
9	019-90319468	JGqFVS	jqgfvss@example.com	Rajshahi	45	Male	Widowed	Graphic Designer	33582	Other	Bangladeshi							
10	019-12211601	lHkXzmtrPR	lhkxmtrpr@example.com	Barisal	42	Male	Divorced	Project Manager	106264	Islam	Bangladeshi							
11	018-18145780	pBtBQCTtIje	pbtbtqcttje@example.com	Khulna	43	Male	Divorced	Software Engineer	97069	Islam	Bangladeshi							
12	010-36433035	RgBEcE	rgbeec@example.com	Rajshahi	37	Female	Divorced	Software Engineer	22024	Islam	Bangladeshi							
13	019-41967475	jHfUflw	jhfufiw@example.com	Rajshahi	43	Female	Married	Marketing Specialist	41333	Buddhism	Bangladeshi							
14	017-85350886	BqtdBkO	bqtdbkko@example.com	Chittagong	48	Female	Single	Project Manager	125574	Hinduism	Bangladeshi							
15	017-13565261	lhvPEbndyB	lhvpebndyb@example.com	Chittagong	29	Female	Married	Marketing Specialist	147415	Buddhism	Bangladeshi							
16	015-5395924	yoySX	yoyrsx@example.com	Chittagong	26	Male	Single	Graphic Designer	124757	Christianity	Bangladeshi							
17	019-62778493	HoVvv	hovvv@example.com	Sylhet	54	Male	Divorced	Software Engineer	119738	Other	Bangladeshi							
18	017-76246270	SRUGzQ	srugzq@example.com	Chittagong	52	Female	Widowed	Project Manager	46606	Buddhism	Bangladeshi							
19	015-74302541	Zytdxhpz	zytdxhpz@example.com	Dhaka	52	Female	Divorced	Graphic Designer	127683	Other	Bangladeshi							
20	015-82416023	JUoGweIK	juogweik@example.com	Dhaka	45	Female	Married	Software Engineer	65932	Buddhism	Bangladeshi							
21	016-42897864	ldwzwEitHz	ldwzweithz@example.com	Sylhet	48	Female	Divorced	Marketing Specialist	71068	Hinduism	Bangladeshi							
22	016-69869738	ifmwkd	ifmwkd@example.com	Barisal	34	Male	Divorced	Software Engineer	146586	Christianity	Bangladeshi							
23	016-87947918	ucbOZu	ucbbozu@example.com	Sylhet	51	Male	Divorced	Graphic Designer	29225	Christianity	Bangladeshi							
24	016-67276094	etTzJIED	ettzjied@example.com	Rajshahi	61	Female	Widowed	Graphic Designer	107359	Islam	Bangladeshi							
25	018-66259488	tbAsYfOp	tbasyfop@example.com	Dhaka	58	Female	Married	Marketing Specialist	110880	Islam	Bangladeshi							

Figure 1.1: Screenshot of the dataset.

As I am going to use Clingo to find the optimal way to guarantee k-anonymity, the data needs to be converted in a set of Clingo statements. The code to convert the data to Clingo statements is in [Listing 1.1](#).

Listing 1.1: Code to convert pandas dataframe to Clingo data

```
1 f = open("lp-files/generated/data/data.lp", "w")
2 for ind in df.index:
3     f.write("person("+str(ind)+").")
4     f.write("\n")
5     for col in df.columns:
6         col_value = str(df[col][ind]).replace(" ", "_")
7         col_value = col_value.lower()
8         f.write("p_"+col+"("+str(ind)+"," + col_value +").")
9         f.write("\n")
10    d[col].add(col_value)
11 f.close()
```

The generated data.lp file looks like in [Listing 1.2](#).

Listing 1.2: Clingo code representing data

```
1 person(27632).
2 p_sex(27632,male).
```

```

3 p_married_status(27632,widowed) .
4 p_job(27632,software_engineer) .
5 p_religion(27632,islam) .
6 person(36119) .
7 p_sex(36119,male) .
8 p_married_status(36119,divorced) .
9 p_job(36119,graphic_designer) .
10 p_religion(36119,buddhism) .
11 ...

```

What also needs to be generated is [Listing 1.3](#). Even though a lot more lp-files need to be generated for the final solution. Especially since the solution should work for every dataset, also for datasets having differently named columns too. But for the moment the files mentioned here are enough to get started.

Listing 1.3: Clingo code representing columns

```

1 column(sex) .
2 column(married_status) .
3 column(job) .
4 column(religion) .

```

1.2 User data

While the previous data was to be generated by the program, the column priorities are to be set by the user. The [Listing 1.4](#) represents some priority ranking for the different columns, so the column that is assigned a 1 will be the first column to be deleted to reach k-anonymity. Columns that are assigned equal numbers have equal priorities. In the end the program should prefer to delete the column that will raise k-anonymity to a value above some chosen value or at least add the k-value for a generated dataset.

Listing 1.4: Clingo code representing priorities set by user

```

1 priority(job,1) .
2 priority(religion,2) .
3 priority(married_status,2) .
4 priority(sex,3) .

```

Chapter 2

Towards a solution

2.1 General idea

The equal column priorities complicate the way to a solution, especially when there are multiple equal priorities. Imagine a problem where there are multiple columns that have priority 2 and also multiple columns that have priority 5.

The solution I will produce in this writing is tested for a simple example as in [Listing 1.4](#), but it should work for the complex case too.

The proposed solution will generate all priorities and the number of columns that have this priority, it will also generate a priority number and a sub priority number for every column present. The full priority numbers represent a deletion rank.

Also all sorts of combinations for the columns will be considered, so these combinations will also have to be generated.

2.2 Generate column priority counts and all possible priority and sub-priority numbers

[Listing 2.1](#) shows the Python-code to generate the priority and sub priority numbers and the priority counts, a count for every priority.

In our case there are 2 columns having a priority of 2, so the priority count is (2,2). the first 2 stands for the priority and the second 2 represents the number of columns having this priority. This priority count will be used to generate all fully qualified priority numbers.

Listing 2.1: Python code to generate priority counts and priorities and sub-priorities.

```
1 def generate_priority_counts():
2     count = 0
3     control = clingo.Control()
4     control.add("base", [], "")
5     control.load("lp-files/fixed-solution/gen-priority-counts-numbers.lp")
6     control.load("lp-files/priorities/col-priorities.lp")
7     control.load("lp-files/fixed-solution/count-cols-priorities.lp")
8     control.load("lp-files/generated/solution/columns.lp")
9     control.add('#show priority_count/2.#show priority_number/2.')
10    control.ground([("base", [])])
11    control.configuration.solve.models = 0
12
13    f = open("lp-files/priorities/generated/priority-counts.lp", "w")
```

```

14 for model in control.solve(yield_=True):
15     for atom in model.symbols(shown=True):
16         f.write(str(atom) + ".\n")
17

```

2.2.1 Clingo-file

The code on lines 5-9 in [Listing 2.1](#) refers to Clingo-files that result in the code in [Listing 2.2](#). The code contains names of files included. Also some comments are present too. Comments that give some idea of the function, the statements that follow, have.

Listing 2.2: Python code to generate priority counts and full priority numbers.

```

1 #columns.lp
2 column(sex) .
3 column(married_status) .
4 column(job) .
5 column(religion) .
6 #col-priorities.lp
7 priority(job,1) .
8 priority(religion,2) .
9 priority(married_status,2) .
10 priority(sex,3) .
11 #count-cols-priorities.lp
12 # the number of priorities are counted
13 # the number f columns are counted
14
15 num_p(NUM_P) :- NUM_P=#count{Y:priority(X,Y)} .
16 num_c(NUM_C) :- NUM_C=#count{X:priority(X,Y)} .
17 #gen-priority-counts-numbers.lp
18 # from the priorities set by the user priority counts are generated per priority
19 # from the priority counts priority and sub priority numbers are generated
20
21 NUM_P {step(1..NUM_P)} NUM_P :- num_p(NUM_P) .
22
23 priority_count(S, C) :- C=#count{X:priority(X,S), column(X), step(S)}, step(S) .
24
25 priority_number(S,K) :- priority_count(S,T), K= 1..T.
26
27 #show priority_count/2.
28 #show priority_number/2.

```

Result

The code in [Listing 2.2](#) results in the facts shown in [Listing 2.3](#).

Listing 2.3: Facts representing priority counts and full priority numbers.

```

1 priority_count(1,1) .
2 priority_count(2,2) .
3 priority_count(3,1) .
4 priority_number(1,1) .
5 priority_number(2,1) .
6 priority_number(2,2) .
7 priority_number(3,1) .

```

2.3 Generate column combinations

The Python code to generate all column combinations is shown in [Listing 2.4](#). The code on lines 2-16 possibly results in multiple answer sets, so for every model a new Clingo-control is created (line 18) to add

a combination id to associated column and priority combinations (line 19-29). I am not entirely satisfied with the creation of a new control, and can think of some alternatives. But I didn't manage to get them to work.

Alternatives for creating a Clingo control for every model

Using `model.context.add_clause` The context for every model has an `add_clause` method, but it isn't exactly clear what to do after adding a clause. This part of the Python Clingo-api is quite obscure.

Using the `#domain` The grouping and naming of combinations of full priorities, columns names possibly could have been reached in an easier way by using the `#domain` statement somewhere in my Clingo-files. This statement makes it possible to add statements for specific models, if I recall it properly.

As the alternatives didn't result in a positive outcome. I had hoped to use generated facts as a direct input for the model-level control, but eventually I had to do it using an old fashioned loop and add construction (lines 22-23).

Lines 32-37 generate a disjunctive string, this string will be added to the resulting file (line 38).

Listing 2.4: Python code to generate column combinations.

```
1 def generate_column_combinations():
2     control = clingo.Control()
3     control.add("base", [], "")
4     control.load("lp-files/priorities/generated/priority-counts.lp")
5     control.load("lp-files/fixed-solution/gen-column-combinations.lp")
6     control.load("lp-files/fixed-solution/count-cols-priorities.lp")
7     control.load("lp-files/priorities/col-priorities.lp")
8     control.load("lp-files/generated/solution/columns.lp")
9     control.ground([("base", [])])
10    control.configuration.solve.models = 0
11    count = 1
12
13    f = open("lp-files/generated/solution/col-combs.lp", "w")
14    comb_str = ""
15
16    with (control.solve(yield_=True) as handle):
17        for model in handle:
18            model_control = clingo.Control([("-c", "comb_n="+str(model.number))])
19            model_control.add("base", [], "")
20            model_control.add("comb(col_priority(X1,S,T),comb_n) :- col_priority(X1,S,T).")
21            model_control.add("#show comb/2.")
22            for atom in model.symbols(shown=True):
23                model_control.add(str(atom)+".")
24            model_control.ground([("base", [])])
25            model_control.configuration.solve.models = 0
26            with model_control.solve(yield_=True) as modelhandle:
27                for model_ in modelhandle:
28                    for atom in model_.symbols(shown=True):
29                        f.write(str(atom)+".\n")
30                count=count+1
31
32    for y in range(1, count):
33        comb_str += "comb(" + str(y) + " ) "
34        if y +1== count:
35            comb_str += "."
36        else:
37            comb_str += "|"
38    f.write(comb_str)
39    f.close()
```

2.3.1 Clingo file

Eventually, after all adding and loading is done, the first Clingo-control (line 2 of [Listing 2.4](#)) needs to interpret [Listing 2.5](#). The code generates all combinations of columns and priorities and sub priorities that contain exactly four columns (line 14). This generation statements uses the number of sub-priorities that exist for every priority to generate the priority numbers. It then removes answers that contain different columns for which the same numbering exists (line 16) and answers for which the same columns and main priority exists having different sub priority numbering (line 15).

Listing 2.5: Clingo code to generate column combinations.

```
1 priority(job,1) .
2 priority(religion,2) .
3 priority(married_status,2) .
4 priority(sex,3) .
5
6 column(sex) .
7 column(married_status) .
8 column(job) .
9 column(religion) .
10
11 num_p(NUM_P) :- NUM_P=#count{Y:priority(X,Y)} .
12 num_c(NUM_C) :- NUM_C=#count{X:priority(X,Y)} .
13
14 NUM_C {col_priority(X, S, K) : priority(X,S), priority_count(S,C), K=1..C} NUM_C :-
    ↪ num_c(NUM_C) .
15 :- col_priority(X1,S1,T1), col_priority(X,S,T), column(X), column(X1), X1=X,S=S, T1
    ↪ !=T .
16 :- col_priority(X1,S1,T1), col_priority(X,S,T), column(X), column(X1), X1!=X,S1=S,
    ↪ T1=T .
17
18 #show col_priority/3.
```

2.3.2 Facts generated

The col-combs.lp file created (on line 13 of [Listing 2.4](#)) looks like in [Listing 2.6](#). It contains the column combinations, full priority number and the combination id . It is clear that in one column combination religion will be obscured before married status while in the other it will be other way around.

As more columns have more equal priorities, combinations grow fast, while their result will not differ that much, as columns with equal priority will be deleted all before a higher priority column will be deleted.

Every combination has a unique id. So it is possible to choose a combination. It should be quiet clear how these facts can be used in a next step. The last line of code represents a disjunctive statement. Either comb(1) is used or comb(2).

Listing 2.6: Priority numbers, columns and combination ids.

```
1 comb(col_priority(job,1,1),1) .
2 comb(col_priority(religion,2,1),1) .
3 comb(col_priority(married_status,2,2),1) .
4 comb(col_priority(sex,3,1),1) .
5 comb(col_priority(job,1,1),2) .
6 comb(col_priority(married_status,2,1),2) .
7 comb(col_priority(religion,2,2),2) .
8 comb(col_priority(sex,3,1),2) .
9 comb(1) | comb(2) .
```

2.3.3 Remark

When using a slightly more complicated example with more equal priorities ([Listing 2.7](#)) and more columns ([Listing 2.8](#)), 4 column combinations are generated ([Listing 2.9](#)), which seems logical as the two columns for each equal priority column generate two possible priority choices. This example isn't explored any further.

Listing 2.7: Column priorities.

```
1 priority(job,1) .
2 priority(religion,2) .
3 priority(married_status,2) .
4 priority(sex,3) .
5 priority(city,4) .
6 priority(street,4) .
7 priority(study,5) .
```

Listing 2.8: Columns.

```
1 column(sex) .
2 column(married_status) .
3 column(job) .
4 column(religion) .
5 column(city) .
6 column(street) .
7 column(study) .
```

Listing 2.9: Column priority combinations.

```
1 comb(del_adv(job,1,1),1) .
2 comb(del_adv(religion,2,1),1) .
3 comb(del_adv(married_status,2,2),1) .
4 comb(del_adv(sex,3,1),1) .
5 comb(del_adv(city,4,2),1) .
6 comb(del_adv(street,4,1),1) .
7 comb(del_adv(study,5,1),1) .
8 comb(del_adv(job,1,1),2) .
9 comb(del_adv(religion,2,2),2) .
10 comb(del_adv(married_status,2,1),2) .
11 comb(del_adv(sex,3,1),2) .
12 comb(del_adv(city,4,2),2) .
13 comb(del_adv(street,4,1),2) .
14 comb(del_adv(study,5,1),2) .
15 comb(del_adv(job,1,1),3) .
16 comb(del_adv(religion,2,1),3) .
17 comb(del_adv(married_status,2,2),3) .
18 comb(del_adv(sex,3,1),3) .
19 comb(del_adv(city,4,1),3) .
20 comb(del_adv(street,4,2),3) .
21 comb(del_adv(study,5,1),3) .
22 comb(del_adv(job,1,1),4) .
23 comb(del_adv(religion,2,2),4) .
24 comb(del_adv(married_status,2,1),4) .
25 comb(del_adv(sex,3,1),4) .
26 comb(del_adv(city,4,1),4) .
27 comb(del_adv(street,4,2),4) .
28 comb(del_adv(study,5,1),4) .
29 comb(1) | comb(2) | comb(3) | comb(4) .
```

2.4 Add a deletion rank to full priority numbers

The full priority numbers and column combinations as generated previously should be extended with some kind of deletion rank, this rank will be used to select all of the columns that will be deleted when deleting some unknown number of columns. The code to generate this file is in [Listing 2.10](#).

Listing 2.10: Python code to add deletion rank to full priority numbers.

```
1 def add_deletion_rank():
2     control = clingo.Control()
3     control.add("base", [], "")
4     control.load("lp-files/fixed-solution/add-deletion-rank-priority-numbers.lp")
5     control.load("lp-files/priorities/col-priorities.lp")
6     control.load("lp-files/fixed-solution/count-cols-priorities.lp")
7     control.load("lp-files/priorities/generated/priority-counts.lp")
8     control.ground([("base", [])])
9     control.configuration.solve.models = 0
10
11     f = open("lp-files/generated/solution/priority-deletion-rank.lp", "w")
12     for model in control.solve(yield_=True):
13         for atom in model.symbols(shown=True):
14             f.write(str(atom) + ".\n")
15     f.close()
```

2.4.1 Clingo-file

The Clingo-file to be interpreted by the control created on line 2 of [Listing 2.10](#) looks like in [Listing 2.11](#). The code adds a number for every priority and sub priority combination. The listing contains describing comments for parts of the code.

Listing 2.11: Clingo code to add deletion rank to full priority numbers.

```
1 num_p(NUM_P) :- NUM_P=#count{Y:priority(X,Y)}.
2 num_c(NUM_C) :- NUM_C=#count{X:priority(X,Y)}.
3
4 priority(job,1).
5 priority(religion,2).
6 priority(married_status,2).
7 priority(sex,3).
8
9 priority_count(1,1).
10 priority_count(2,2).
11 priority_count(3,1).
12 priority_number(1,1).
13 priority_number(2,1).
14 priority_number(2,2).
15 priority_number(3,1).
16
17 NUM_C {deletion_rank(priority_number(S,T), 1..NUM_C): priority_number(S,T)} NUM_C :-
18     ↪ num_c(NUM_C).
19
20 % no same full priority numbers should have a different rank number
21 :- deletion_rank(priority_number(S,T),X), deletion_rank(priority_number(S,T),Y), X!=
22     ↪ Y.
23
24 % priority numbers that have the same main priority number and a subpriority number
25     ↪ only differing 1, should have a rank number that differs only 1.
26 :- deletion_rank(priority_number(S,T),X), deletion_rank(priority_number(S,T+1),Y), Y
27     ↪ -X!=1.
28
29 % minimum rank should be 1
30 :- #min{X:deletion_rank(priority_number(_,_),X)}!=1.
31
32 % minimum main priority number should be 1
33 :- #min{X:deletion_rank(priority_number(X,_),_)}!=1.
34
35 % rank number 1 should only be given to a main priority number 1
```

```

27 :- deletion_rank(priority_number(S,_),1), S!=1.
28 % no two different main priority numbers can have the same rank number
29 :- deletion_rank(priority_number(S,_),X), deletion_rank(priority_number(S1,_),X), S
    ↪ !=S1.
30 % higher main priority number should have a rank number that is higher too
31 :- deletion_rank(priority_number(S,_),X), deletion_rank(priority_number(S1,_),X1), S
    ↪ >S1, X<X1.
32
33 #show deletion_rank/2.

```

2.4.2 Facts generated

The priority-deletion-rank.lp file created looks like in [Listing 2.12](#). It contains the full priority numbers and a deletion rank for that particular priority number. It should be clear that when deleting 2 columns, the following full priority numbers are to be selected: 1,1 and 2,1. these numbers will be used to select the columns in the different combinations in [Listing 2.6](#).

Listing 2.12: Priority numbers and column counts.

```

1 deletion_rank(priority_number(1,1),1).
2 deletion_rank(priority_number(2,1),2).
3 deletion_rank(priority_number(2,2),3).
4 deletion_rank(priority_number(3,1),4).

```

Chapter 3

Searching for the solution

3.1 Purpose

In the previous chapter column combinations and their respective priority and sub priority numbers were generated, also a deletion rank was created for every full priority combination. Next to that the data (in Clingo statements) was generated from a csv-file. So in this chapter I will describe the process of searching for the most optimal k-anonymous complying data column combination that can be found.

3.2 Remarks

Some Clingo-files that will be used in this chapter will also have been generated from the csv-dataset. I will not show the Python code to generate these data specific Clingo-files.

As a matter of fact it could be the case these data specific files could perhaps be rewritten in a more generic way so that no association between data and columns names is used. But this will not be a part of the writing at hand.

On line 2 of [Listing 3.1](#) an `--opt-mode=optN` parameter is used , I believe this argument isn't really necessary.

3.3 Searching

The code in [Listing 3.1](#) will combine the `data.lp` file (containing personal data), the `col-combs.lp` file (containing the column and priority combinations), the `priority-deletion-rank.lp` file (containing priority numbers and deletion ranks) and the `columns.lp` file (containing columns information). It also uses some new files which will be described later.

The code on line 7-15 in [Listing 3.1](#) generates:

```
del_c(0)|del_c(1)|del_c(2)|del_c(3).
```

This statement expresses the number of columns to delete. This number will be minimized by using a statement like the following:

```
#minimize {X:del_c(X)}.
```

In the end the combination of the Clingo files will generate data files for every column deletion combination that has a k-anonymity of 4 or more (line 1), at the same time it will minimize the number of columns deleted, but will generate a `data.lp` file ,even if more columns than the minimum are deleted, as long as it conforms to the required k-anonymity value.

It will rewrite the data.lp to a/multiple new data.lp file(s) that will contain 'na' for the columns obscured by the search algorithm. The generated file will also contain some metadata on the k-value, the number of columns deleted,....

The generated data files will be transformed to csv-files in a following section.

Listing 3.1: Clingo code to generate data that is at least k-anonymous.

```

1 def generate_k_anonym_data():
2     control = clingo.Control(["--opt-mode=optN", "-c", "k=4"])
3     control.add("base", [], "")
4     control.load("lp-files/generated/data/data.lp")
5     control.load("lp-files/generated/solution/col-combs.lp")
6
7     del_c_str = "del_c(0) | "
8     for h in range(1, num_col_data() - 2):
9         if h != num_col_data():
10            del_c_str += "del_c(" + str(h) + " ) "
11            del_c_str += " | "
12            del_c_str = del_c_str[:-1]
13            del_c_str += "."
14            print(del_c_str)
15            control.add(del_c_str)
16
17     control.load("lp-files/generated/solution/columns.lp")
18     control.load("lp-files/generated/solution/priority-deletion-rank.lp")
19     control.load("lp-files/generated/solution/search-constraints.lp")
20     control.load("lp-files/generated/solution/col-specific-hide.lp")
21     control.load("lp-files/generated/solution/values.lp")
22     control.ground(["base", []])
23     control.configuration.solve.models = 0
24     count = 1
25
26     for model in control.solve(yield_=True):
27         if model.optimality_proven:
28             f = open("lp-files/generated/data-to-be-exported/data-" + str(count) + ".lp", "w
29                 ↪ ")
30             for atom in model.symbols(shown=True):
31                 f.write(str(atom) + ".\n")
32             f.close()
33             count = count + 1

```

3.3.1 Clingo file

The Clingo file to be interpreted by the control created on line 2 of [Listing 3.1](#) is available in an external file named complete_solution.lp as it contains too many statements to be useful in this writing. It is useful however to check the results.

One should use a command like:

clingo complete_solution.lp -c k=4

Keep in mind one could change k to become a higher (when raising k) or lower (when lowering k) minimum degree of privacy for the individuals in the data file.

3.3.2 Data specific files

Some files used in [Listing 3.1](#) referred to files that were generated when generating the personal data from the csv-file. They contain data specific statements.

values.lp

The values for every column in the dataset are converted into a Clingo file that is shown in [Listing 3.2](#), values.lp. For every column 'na' is added too. Without this data.lp file counting of the column combinations will not work or will be wrong.

Listing 3.2: Values used in the dataset.

```
1 sex(female) .
2 sex(male) .
3 sex(na) .
4 married_status(divorced) .
5 married_status(single) .
6 married_status(married) .
7 married_status(widowed) .
8 married_status(na) .
9 job(software_engineer) .
10 job(accountant) .
11 job(graphic_designer) .
12 job(marketing_specialist) .
13 job(project_manager) .
14 job(na) .
15 religion(other) .
16 religion(buddhism) .
17 religion(christianity) .
18 religion(hinduism) .
19 religion(islam) .
20 religion(na) .
21 val(na) .
```

3.3.3 col-specific-hide.lp

In [Listing 3.1](#) a reference was made to the col-specific-hide.lp file, this file looks like [Listing 3.3](#). In this code counts are made for every value combination of the four columns in our dataset (line 9). Column value combinations that have no individuals belonging to it, are filtered out (line 10). In line 11 counts for all column combinations are summed up to become the number of rows in the dataset. Line 12 generates the number that is equal to the fewest number of individuals belonging to a column value combination, k should be lower or equal to this number to comply to a k-anonymity requirement.

Once a column gets hidden, its value becomes 'na' (line 1,3,5,7) for every row in the column. And in this way individuals become more and more anonymous. If columns aren't to be hidden the data will not be obscured (line 2,4,6,8). Mind the added _ to the head of the statement stating a fact about a person like on line 3, where p_married_status becomes p_married_status_.

Listing 3.3: Hide columns or not, counting combinations, testing combination counts.

```
1 p_sex_(X,na) :- p_sex(X,_), hide_column(sex,_).
2 p_sex_(X,Y) :- p_sex(X,Y), not hide_column(sex,_).
3 p_married_status_(X,na) :- p_married_status(X,_), hide_column(married_status,_).
4 p_married_status_(X,Y) :- p_married_status(X,Y), not hide_column(married_status,_).
5 p_job_(X,na) :- p_job(X,_), hide_column(job,_).
6 p_job_(X,Y) :- p_job(X,Y), not hide_column(job,_).
7 p_religion_(X,na) :- p_religion(X,_), hide_column(religion,_).
8 p_religion_(X,Y) :- p_religion(X,Y), not hide_column(religion,_).
9 count_ext(X0,X1,X2,X3,N) :- sex(X0), married_status(X1), job(X2), religion(X3), N=#count{
    ↪ X: p_sex_(X,X0), p_married_status_(X,X1), p_job_(X,X2), p_religion_(X,X3)}.
10 count_ext_(X0,X1,X2,X3,N) :- count_ext(X0,X1,X2,X3,N), N!=0.
11 sum_test(N1) :- N1=#sum{N,X0,X1,X2,X3 : count_ext(X0,X1,X2,X3,N)}.
12 min(N1) :- N1=#min{N:count_ext_(X0,X1,X2,X3,N)}.
```


3.3.4 search-constraints.lp

In [Listing 3.1](#) a reference was made to the search-constraints.lp file, this file looks like [Listing 3.4](#). But before delving into the code, a quick reminder on disjunctive statements used.

Disjunctive statements

The disjunctive statement that allowed the selection the number of columns to be deleted looks like:

del_c(0)|del_c(1)|del_c(2)|del_c(3).

This statement was added using Python code, see [Listing 3.1](#).

In [Listing 2.6](#) multiple column combinations were generated plus a disjunctive statement:

comb(1)|comb(2).

By using this disjunctive statement it is assured only one combination will be used to generate the possibly adapted data, the resulting personal data will be generated depending on the column deletion rank used for this specific combination.

Priority deletion selection

The statement on line 1 of [Listing 3.4](#) will select the full priority numbers that have a deletion rank equal to or lower than the number of columns deleted (del_c(S)). The deletion rank file looked like [Listing 2.12](#). The priority numbers to be deleted are queued for deletion so to speak.

Column selection

Line 2 assures that only columns (origination from the selected combination) associated with a priority number (included in the queue created in line 1) will be used to generate a column deletion for a specific column combination.

Hide column statements

Explicit hide_column statements are generated for specific combinations in line 4, these statements are used in [Listing 3.3](#) to hide the columns.

K-value constraint

The constraint that checks whether the minimum number of individuals counted for a combination is higher than k is on line 6.

Minimize del_c

The minimize statement that forces the number of column deletions to a minimum is on line 8.

Other statements

On line 12-16 it is assured the (possibly deleted) data is shown. Other statements are related to data describing the data.

Listing 3.4: Search the data for right combinations of column priority numbers and deletion ranks complying to a k-value, while minimizing the number of deleted columns.

```
1 queue(priority_number(X,Y),CN) :- deletion_rank(priority_number(X,Y),CN), CN<=S,  
  ↪ del_c(S).  
2 col_priority_comb(C,X,Y,V) :- queue(priority_number(X1,Y1),CN), comb(col_priority(C,  
  ↪ X,Y),V), comb(V), column(C), X=X1, Y=Y1.
```

```

3
4 hide_column(C,N) :- col_priority_comb(C,X,Y,N), comb(N).
5
6 :- k>N1, min(N1).
7
8 #minimize {X:del_c(X)}.
9
10 #show hide_column/2.
11 #show comb/1.
12 #show person/1.
13 #show p_religion_/2.
14 #show p_job_/2.
15 #show p_sex_/2.
16 #show p_married_status_/2.
17 #show val/1.
18 #show sum_test/1.
19 #show min/1.
20 #show del_c/1.

```

3.3.5 Facts generated

The facts generated are listed in [Listing 3.5](#), it includes the combination used (line 1), the number of columns deleted (line 2), which columns are deleted (line 3 and 4), the minimum count of individuals when comparing all combinations, excluding 0 (line 5). For line 3 and 4 the second argument is the combination number so the same as used on line 1.

And furthermore the file contains the facts needed to generate a possibly partially obscured csv-dataset. As mentioned before it is possible multiple data files are generated of which the one having a higher min value is more anonymous.

Listing 3.5: Priority numbers and column counts.

```

1 comb(1).
2 del_c(2).
3 hide_column(job,1).
4 hide_column(married_status,1).
5 min(4).
6 p_sex_(27632,male).
7 p_sex_(36119,male).
8 p_sex_(4796,female).
9 ...
10 p_married_status_(16979,na).
11 p_married_status_(27534,na).
12 p_married_status_(19130,na).
13 p_married_status_(9957,na).
14 ....
15 p_job_(21255,na).
16 p_job_(20325,na).
17 p_job_(14324,na).
18 p_job_(19186,na).
19 ...
20 p_religion_(14324,other).
21 p_religion_(19186,christianity).
22 p_religion_(23373,other).
23 p_religion_(2866,islam).
24 ...
25 person(19186).
26 person(23373).
27 person(2866).
28 person(12496).
29 ...

```

3.4 Export to csv

In [Listing 3.6](#) the code used to generate the csv-file is shown, lines 1-11 will generate a list of ids for the persons, after which the column values for every person are fetched and written to csv (line 13-29).

Listing 3.6: Python code to generate csv-file from Clingo file .

```
1 def generate_csv_from_asp():
2     ids = []
3     control = clingo.Control()
4     control.add("base", [], "")
5     control.load("lp-files/generated/data-to-be-exported/data-4.lp")
6     control.add("#show person/1.")
7     control.ground([("base", [])])
8     control.configuration.solve.models = 0
9     for model in control.solve(yield_=True):
10         for atom in model.symbols(shown=True):
11             ids.append(atom.arguments[0])
12
13     f = open("lp-files/generated/data-to-be-exported/data-4.csv", "w")
14     for i in ids:
15         arg = "id=" + str(i)
16         item_control = clingo.Control(["-c", arg])
17         item_control.load("lp-files/generated/data-to-be-exported/data-4.lp")
18         item_control.load("lp-files/generated/solution/get-item.lp")
19         item_control.load("lp-files/generated/solution/values.lp")
20         item_control.ground([("base", [])])
21         item_control.configuration.solve.models = 0
22         f.write(str(i))
23         f.write(";")
24         for model in item_control.solve(yield_=True):
25             for atom in model.symbols(shown=True):
26                 f.write(str(atom.arguments[1]))
27                 f.write(";")
28             f.write("\n")
29     f.close()
```

3.4.1 New files referred to

[Listing 3.6](#) refers to get-item.lp, the contents of this file is to be found in [Listing 3.7](#). The code fetches the data related to a person one person at a time. Some constant argument is added in line 16 of [Listing 3.6](#), this constant argument selects the person for which data is fetched. The code in this get-item.lp file is generated from the original csv file.

Listing 3.7: Clingo code to fetch columns related to that person .

```
1 person_(X) :- person(X), X=id.
2 p_sex_out(X, Y) :- p_sex(X, Y), X=id.
3 p_married_status_out(X, Y) :- p_married_status(X, Y), X=id.
4 p_religion_out(X, Y) :- p_religion(X, Y), X=id.
5 p_job_out(X, Y) :- p_job(X, Y), X=id.
6
7 #show p_sex_out/2.
8 #show p_job_out/2.
9 #show p_religion_out/2.
10 #show p_married_status_out/2.
```